KENEXOFT · TECHNICAL DOCUMENT

Continuous Trip Risk Monitoring

Architecture, modules and code walkthrough of the Arte Voyages trip risk monitor

AUTHOR
Charanjeeth Shasta Pendam
Software and AI Engineering Intern
(Technical Architect)

CLIENT
Kenexoft Technologies Pvt Ltd

DATE AND STACK
7 November 2026
Python 3.12 · Streamlit · SQLite / Turso

1 Abstract

Tour operators often hear about disasters from the news, after clients are already affected. This system polls free hazard feeds every 30 minutes, scores every trip leg (a city and its travel dates) as **Green**, **Watch** or **Act**, and emails a plain-text alert only when a tier changes. A read-only Streamlit dashboard shows the live state, and a Manual mode runs what-if simulations without touching live data.

2 Objectives

Continuous awareness
Polls feeds every 30 min and scores every trip leg.

Explainable scoring
Four visible factors per score, so the driver is clear.

Low noise
Alerts on tier change only; 48 h cooldown on the way down.

Reliable at $0
A failing feed never blocks the others; no paid API keys.

3 System architecture

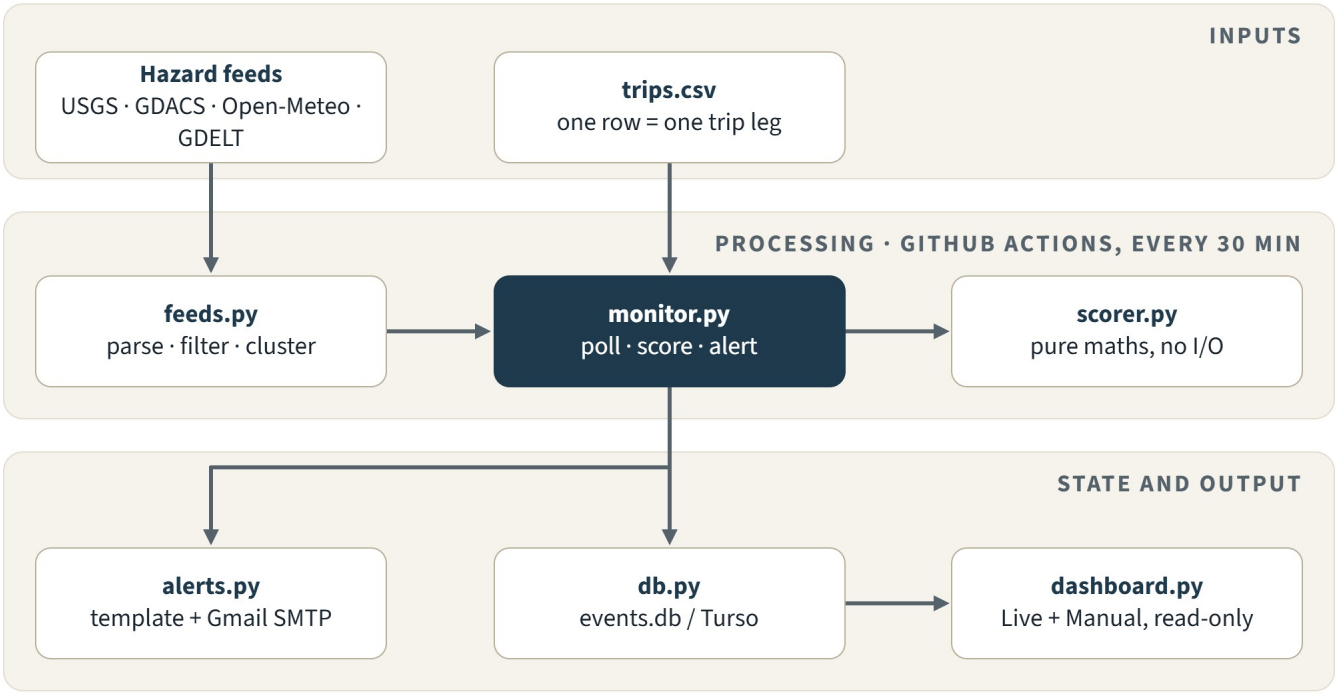


Figure 1 — The monitor writes to the database; the dashboard only reads it. scorer.py is shared with Manual mode.

4 Modules

Six Python files and one CSV. Each file has one job, and the scoring maths is isolated from all network and database code.

MODULE	RESPONSIBILITY	DESIGN CHOICE
`scorer.py`	Leg and Event dataclasses, distance, proximity, time overlap, noisy-OR, tiers, manual simulation	No I/O, so it is easy to unit-test and is reused by the dashboard
`feeds.py`	Five parsers, four network wrappers, scope filter, geocoding, clustering	Parsers are pure and tested offline; wrappers isolate the network
`monitor.py`	Loads trips, runs one poll tick, applies the cooldown, decides alerts	Stateless process; all state lives in the database
`db.py`	Four tables, three indexes, idempotent migrations, upsert and read helpers	A file: URL uses sqlite3, libsql:// uses Turso; same SQL
`alerts.py`	Template alert text and Gmail sender, no LLM	Deterministic, auditable messages
`dashboard.py`	KPIs, score table, map, 14-day outlook, alert feed, feed health	Presentation only; never calls feeds, never writes

5 Data sources

All sources are free and need no API key. Each parser turns an item into an Event with a severity and a confidence.

SOURCE	PROVIDES	BECOMES AN EVENT
USGS earthquakes	Real-time quakes, M 4.5+ weekly GeoJSON feed	Keep M 5 and above. Severity = min(1, (M − 4.5) / 4): M6 0.38, M7 0.63, M8 0.88. Confidence 0.9, active 7 days.
GDACS	Quakes, cyclones, floods, volcanoes, drought, wildfire	Alert level Green 0.2, Orange 0.55, Red 0.9. Confidence 0.9; duration taken from the feed.
Open-Meteo	14-day forecast of gusts, rain and heat for each leg city	Gust 70 / 90 km/h, rain 50 / 100 mm, heat 40 / 42 °C map to 0.55 / 0.9. Confidence 0.6, because it is a forecast.
GDELT and ReliefWeb	News headlines	Keywords set severity 0.8 / 0.55 / 0.35. The city is found by matching the title to GeoNames, with a rapidfuzz fallback. Confidence 0.6.

Scope and clustering. Five country bounding boxes filter events, small countries first because India's box contains Nepal and Bhutan. Events of the same type within 150 km and 3 days merge into one.

6 Deployment and quality

Deployment
GitHub Actions runs python -m monitor every 30 minutes with five secrets: Turso URL and token, Gmail user, app password and alert recipient. The dashboard runs on Streamlit free hosting.

Testing
36 pytest functions in 9 files cover scorer maths, feed parsers, DB round-trip, alert text, monitor behaviour (alert, bad CSV, no duplicates, email retry) and dashboard helpers.

Quality
Tiers always show colour plus text. WCAG AA contrast, 44 px touch targets and English-only basemap labels.

7 Scoring engine

scorer.py is pure maths. For every leg and every known event, four factors between 0 and 1 are multiplied into a risk; the risks of all events are then combined into one leg score.

Severity	×	Proximity	×	Time overlap	×	Confidence
How bad the hazard is, 0 to 1		exp(−distance / radius), 0 beyond 3× radius		Share of the leg's days the event is active		0.9 measured data, 0.6 forecasts and news

Leg score = 1 − Π (1 − risk$_i$) · noisy-OR: weak signals add up, and the score never exceeds 1

Green < 0.15	Watch 0.15 to 0.45	Act 0.45 and above

```python
# radius per hazard, in km
RADIUS_KM = {"earthquake": 300, "cyclone": 400, "flood": 150, "unrest": 50, ...}

def proximity(event, leg):
    r = RADIUS_KM.get(event.type, RADIUS_KM["default"])
    dist = haversine_km(event.lat, event.lon, leg.lat, leg.lon)
    return 0.0 if dist > 3 * r else math.exp(-dist / r)

def time_overlap(event, leg):
    ev_end = event.date + timedelta(days=event.duration_days)
    lo, hi = max(event.date, leg.start), min(ev_end, leg.end)
    return max(0, (hi - lo).days + 1) / ((leg.end - leg.start).days + 1)

def noisy_or(values):
    p = 1.0
    for v in values:
        p *= 1 - v
    return 1 - p
```

Listing 1 — Core of scorer.py. Tiers come from tier_of(score, watch=0.15, act=0.45).

Proximity. Great-circle (haversine) distance with exponential decay. The radius depends on the hazard: 400 km for a cyclone, 50 km for unrest.

Time overlap. Compares the event window with the leg dates, so a hazard that ends before arrival scores 0.

No look-ahead. Events issued after the evaluation day are skipped. The thresholds are parameters, shown as sliders in Manual mode.

8 Worked example

Leg DELHI-01, 10 to 15 December 2026 (6 days). Simulated earthquake: severity 0.5, 50 km away, starts 10 December, lasts 3 days, confidence 0.9.

Proximity	Time overlap	Risk and score	Tier
0.847	0.667	0.254	Watch
exp(−50 / 300)	4 of 6 days	0.5 × 0.847 × 0.667 × 0.9	Escalated from Green

Table 1 — Matches the Manual-mode card in the dashboard: “Leg risk 0.25 → Watch (was Green)”. With one event the noisy-OR equals the risk.

9 Monitor, cooldown and alerts

Each 30-minute tick fetches the feeds, scores every leg and alerts on a tier change. A leg rises to a higher tier at once, but falls only after 48 hours below.

```
def _tier_with_cooldown(prev_tier, prev_since, raw, now):
    if _TIER_RANK[raw] ⩾ _TIER_RANK[prev_tier]:
        return raw, None                    # rise immediately
    since = prev_since or now
    if now - since ⩾ COOLDOWN:              # 48 h below the old tier
        return raw, None
    return prev_tier, since                 # otherwise holdif tier ≠ prev_tier:
    db.log_alert(a)                         # 1. persist first
    try:
        notify(a)                           # 2. then send the email
    except Exception:                       # failed: do not advance the tier,
        db.update_score(..., prev_tier, ...)  # so the next tick retries
```

Listing 2 — Cooldown and persist-before-notify logic from monitor.py (shortened).

Persist, then notify. The alert row is written before the email, so a mail failure never loses an alert.

Fault isolation. Each feed has its own try/except; a failure goes to feed health and the run continues.

Closed legs. After a leg's end date it is forced to Green with one CLOSED alert.

10 Dashboard: Live and Manual modes

Live mode reads scores, alerts, feed health and events from the database. Manual mode is **simulation only**: staff pick a leg and hazard values, and simulate_manual() recomputes all 8 legs in memory. It never writes to the database and never sends email.

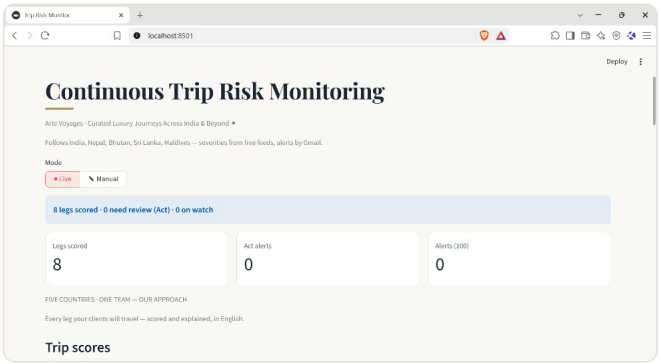


Figure 2 — Live mode: 8 legs scored, none above Green.

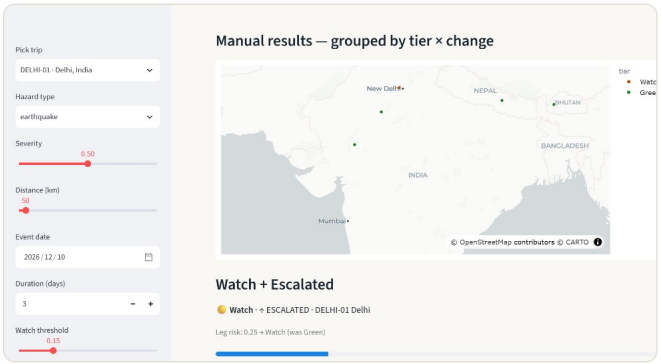


Figure 3 — Manual mode: a simulated earthquake escalates DELHI-01 to Watch.

11 Limitations and next steps

Heuristic weights
Radii, severities, thresholds and confidences are reasoned starting values, not validated accuracy.

Licensing
Open-Meteo's free tier is non-commercial and the GeoNames licence must be confirmed before commercial use.

MVP scope
Single operator, hand-edited CSV. Next: backtest and replay to calibrate weights, authentication, booking-system upload.